InstructGPT 论文问答记录

Training Language Models to Follow Instructions with Human Feedback (arXiv:2203.02155v1)

按论文章节组织，章节内按论文顺序。

---

0 术语与阅读约定

同一英文词根在中文里必须共享词根，否则论文里的推理链会断。

Q：misaligned 为什么不能译成「目标错位」？

A：align 译作「对齐」，misaligned 就必须是「未对齐」，两者是同词根的正反两态。译成「错位」会切断两处推理：alignment tax（对齐税）是「为消除未对齐这一缺陷所付的能力代价」；§5.2「我们在对齐谁」成立的前提是 align 天生要带参照对象（align with whom）。换词根之后这两处都读不通。

Q：honest 和 truthfulness 可以混用吗?

A：不能，论文是刻意区分的。honesty（诚实）关乎意图与信念，要衡量它得比较模型输出和它的「信念」，而模型是黑箱，推不出信念，所以论文放弃测诚实。truthfulness（真实性）只问模型关于世界的陈述是否为真，可测。这是一次明确的术语降级，中文不可混用。

Q：response / output / completion 是三个东西吗?

A：同一个东西，论文在不同位置换用：§3.5 讲 PPO 环境用 response，讲标注和评测用 output，附录 C.2 讲 RM 训练数据用 completion。式 (1)、式 (2) 里的 y 就是它。

Q：demonstration 和 response 呢?

A：这两个必须分开。demonstration 是标注员手写的那一条，用于 SFT；response 是模型生成的。RM 归一化用的是 demonstration 集的均值，不是 RM 训练数据的均值。

---

1 引言与问题设定

Q：论文提出的方法叫什么？InstructGPT 是方法还是模型?

A：InstructGPT 是**模型的名字**，是走完三步之后得到的产物。方法是 RLHF。论文里说的 InstructGPT 默认指 PPO-ptx 那个版本。

Q：RLHF 三步，是 RLHF 包括这三步，还是三步共同组成 RLHF?

A：都不准确。准确的边界是：

- **三步在一起**是本文的方法（后训练流水线）
- 其中**第 2 + 3 步是 RLHF**（第 2 步提供 HF，第 3 步提供 RL）
- 只有**第 3 步是 RL**

SFT 是 RLHF 的初始化，不是 RLHF 的一部分。

Q：helpful / honest / harmless 三个目标都测了吗?

A：只有第一个能直接测，用标注员偏好评分，是主指标。honest 测不了，降级成 truthfulness（闭域幻觉率 + TruthfulQA）。harmless 也测不了，因为危害取决于输出在现实中怎么被使用——项目早期让标注员直接判断「是否潜在有害」，后来放弃了，改用一组具体代理：是否适合客服场景、是否贬损受保护群体、是否含色情或暴力内容。

Q：把模型做大能不能自动解决未对齐?

A：不能。预训练目标是预测网页下一个 token，和「有帮助、安全地遵循用户指令」是两回事，这个缺口不随规模缩小。大模型照样编造事实、输出有毒内容、无视指令。

---

2 相关工作：RLHF 的谱系

Q：最初的 RLHF 是什么？和大模型相关吗?

A：不相关。RLHF 的算法内核出自 Christiano et al. 2017，做的是**模拟机器人与 Atari 游戏**——用人类对两段轨迹的偏好训一个奖励模型，再用 RL 优化它。语言化是后来的事。

年份	工作	做了什么
2017	Christiano et al.	RLHF 内核，模拟机器人 / Atari
2019	Ziegler et al.	首次用于语言模型微调
2020	Stiennon et al.	摘要任务上的 RLHF，本文直接继承其 PPO 配置与 β=0.02
2022	本文	推广到开放指令分布

Q：那本文改进了什么?

A：算法内核不是本文的贡献。本文做的是把 RLHF 从**一个窄任务**推到**开放指令分布**，并补上让它能落地的两块拼图：on-distribution 的真实用户数据，以及把对齐税压下去的预训练梯度混合（ptx）。

算法层面唯一的新增是 ptx 项。

3 方法

3.2 数据集与划分

Q：train / test 是怎么切的？比例多少？

A：按 **user ID** 切，不是按 prompt 随机切，切出 train / validation / test 三份。按客户切是为了防泄漏——同一客户的 prompt 往往高度相似，随机切会让模型在训练时见过几乎一样的东西。

Table 9 给出各份条数：

阶段	train	valid
SFT	12,725	1,653
RM	33,207	17,887
PPO	31,144	16,185
test set（不隶属任何阶段）	3,196	—

以 PPO 阶段算训练:测试约 10:1；三阶段训练 prompt 合计约 77,000，test set 占约 4%。test set 这个数字只出现在 Table 9，正文从未提过。

Q：33,207 是比较对数吗？

A：不是，**单位是 prompt 条数**。Table 6 下方明说：每个 prompt 收集 K 个输出的排序（K 从 4 到 9），在全部 C(K,2) 上训练，所以实际比较对数比 prompt 数**大一个数量级**，约五十万量级。

Q：三个阶段的 prompt 一样吗？

A：不一样，来源构成差别很大。SFT 里标注员自写的占约 89%，RM 里客户的占约 80%，**PPO 则 100% 来自 API**，没有任何标注员自写的 prompt。这个梯度是冷启动造成的：最初没有真实用户，只能让标注员自己写（Plain / Few-shot / User-based 三类）；等早期 InstructGPT 上线后，后面两个阶段就能用真实流量了。

3.5 Step 1 · SFT

Q：SFT 全称是什么？做什么？

A：supervised fine-tuning，在人类 demonstration 上做监督微调。目标是让模型输出尽量接近标注员写的那条标准答案，就是普通的因果语言建模，**只在 response 部分算 loss，prompt 部分掩掉**。

Q：不计入 prompt 的损失，是不是 prompt 部分就没有梯度?

A：不是。梯度仍然会流到 prompt 的位置——被掩掉的是**损失项**，不是反向传播的路径。prompt 的表示仍然参与了 response 位置的预测，所以它照样贡献梯度。单个位置的梯度贡献不变，但参数拿到的是所有位置贡献的加权和，而这个和变了。

Q：配置?

A：16 epoch、cosine 衰减、residual dropout 0.2。验证 loss 在 1 epoch 后就过拟合，但**继续训练反而提升 RM 分数与人类偏好**，所以按 RM 分数选 checkpoint，不按验证 loss。

Q：为什么不直接把 SFT 数据混进预训练?

A：四个原因。量级悬殊会被直接淹掉；顺序有效应，SFT 起作用恰恰因为它在最后；迭代经济学——改 SFT 数据只要重跑几小时，改预训练要重跑几周；以及预训练数据是描述性的、SFT 数据是规范性的，混了就失去「目标」地位。

3.5 Step 2 · RM

Q：RM 是什么？到这里进入 RL 了吗?

A：Reward Model，奖励模型。还没进 RL——第 2 步提供 RLHF 里的 HF，第 3 步才提供 RL。

Q：RM 就是给 PPO 做准备的吗？因为它输出一维标量便于接收?

A：对。人类偏好没法直接当奖励函数，RM 把它编译成一个可以无限次调用的标量打分器，PPO 就不用每步去问人了。

Q：标注员拿到什么？C(K,2) 个对从哪来?

A：标注员拿到 1 个 prompt 和 K 个 response（K=4~9，来自多个策略：多数 SFT、少量 PPO），界面分两页——先对每条单独打 1–7 的 Likert 分并勾 metadata 标签，再做整体排序。排序**只做一次**，C(K,2) 个比较对是机器自动展开的。K=9 时一次标注换 36 条样本。

Q：既然是排序，哪来的平局?

A：排序是**弱序**，不是严格全序。Figure 12(b) 明说：两个输出质量相近时**鼓励**打平。丢掉的是**那一个对**，不是那两个 response——若 y2 与 y3 并列，只丢 (y2,y3)，其余对全部保留。必须丢是因为 Bradley–Terry 损失要求指定谁是 winner 谁是 loser，平局没有。所以实际对数 ⩽ C(K,2)，这就是「batch 64 个 prompt，⩽2,304 个比较对」里那个不等号的来源。

Q：一个 pair 长什么样?

A：有序三元组（`x, y_w, y_l`），**顺序本身就是标签**，没有独立的 label 字段。现代实现里就是 prompt / chosen / rejected 三列。注意「y1 好」永远是**对内相对**的，同一条 response 在不同对里角色会翻转。

实现上 pair 不作为数据存在：K 个 completion 各跑**一次**前向得到 K 个标量，再在这 K 个标量上取 C(K,2) 个差。K=9 时是 9 次前向，不是 36 次。

Q：RM 训完还要做什么?

A：只有一件事——**归一化**。加一个偏置，使**标注员 demonstration** 的平均分为 0（不是 RM 训练数据的均值）。因为 Bradley–Terry 只约束差值，绝对零点是任意的。选 demonstration 集当零点，是因为它是全流程唯一一批人类亲手写的输出，钉在 0 之后 reward 就有了「高于/低于人类示范水平」的含义。这不改变最优策略，价值在可解释性。

Q：为什么用 6B RM 而不是 175B?

A：三条理由。175B RM 训练更不稳定，因而**不适合用作 PPO value function 的初始化**；175B 的 RM 和 value function 会大幅推高 PPO 算力；而 6B 在很宽的学习率范围内都稳定，且得到**同样强**的 PPO 模型。另外附录 C.4 还有一条实验设计上的理由：所有规模的策略共用同一个 6B RM，才好比较策略规模本身的影响。

顺带：这个 6B RM 不是从 SFT 初始化的，而是从一个在若干公开 NLP 数据集上微调过的 6B GPT-3 初始化，论文说这**主要是历史原因**。

3.5 Step 3 · PPO

Q：PPO 全称?

A：Proximal Policy Optimization（近端策略优化），Schulman et al. 2017。命门在 proximal——**新策略必须待在旧策略附近**。

Q：RM 是可微的神经网络，为什么不直接反向传播?

A：中间隔着一次**采样**，从概率分布里抽出一个具体 token 这个操作没有梯度。policy gradient 绕过它——不对 reward 求导，改对 `log π` 求导，reward 只当权重。

Q：PPO 的四步?

A：采样 → 打分 → 算梯度 → 更新。频率不等：第 1、2 步各做一次，第 3、4 步做**八次**（512 切成 8 个 minibatch）。

Q：场上有几个模型？SFT 和 RM 各交付了什么？

A：四个。SFT 交付两样：policy 的初始权重 + 一个**冻结副本**（KL 惩罚的参照）。RM 也交付两样：奖励的第一项来源 + value function 的初始权重。

模型	来自	PPO 期间
policy	SFT 权重	在训
value function	RM 权重复制	在训
RM	Step 2	冻结
SFT 参照副本	SFT	冻结

Q：π 是什么？

A：policy 的数学记号，就是那个语言模型本身。π(token | 上下文) 是模型给这个 token 的概率。整条 response 的概率是每个 token 概率的连乘。π^SFT 是冻结副本，π^RL_φ 是在训的 policy，φ 是它的参数。

Q：什么是 KL 惩罚？从哪来的？

A：**PPO 阶段新增的**，SFT 和 RM 两步都没有——「偏离 SFT」这个概念只有在 policy 开始被改动之后才成立。它也不是 PPO 自带的，PPO 拿去训机器人时没有 SFT 可偏离。论文继承自 Stiennon et al. 2020，β = 0.02。

偏离的是**模型本身**，不是输出。PPO 收到的指令只有「让 RM 分数尽可能高」，没有任何东西让它保持原样。而 RM 是在 SFT 采样出的 response 上训的，策略一旦走远就进入 RM 的**分布外区域**，那里 RM 照样吐一个数，纯属外推。优化器会专门往那里跑，因为分布内的分数被人类排序钉住了，涨不了多少。结果是 **RM 分数一路上涨，回答实际越来越差**——论文称之为 over-optimization。

机制：逐 token 比较 policy 和冻结 SFT 副本给同一个 token 的概率，取对数比值。**双向**——policy 更爱写就扣分，更不爱写就加分，罚的是「不一样」本身。β 是兑换率，不禁止改变，只是**给改变标价**。

Q：β=0.02 是最优解吗？

A：是继承来的，但附录 E.7 扫过：β=0 和 β=2 都很差，最优在 0.01–0.02。这条曲线本身就是 over-optimization 的证据——如果最大化 RM 分总是等于变好，β 就不该有内部最优。注意这个值只对这套配置成立，**RM 输出的数值尺度一变，β 就得重调**。

Q：什么是 value function？是 RM 吗？

A：不是，但它是 **RM 的一个副本**（从 RM 初始化）。它预测「从当前局面出发、照当前 policy 写完，预计能拿多少奖励」。

	RM	value function
输入	prompt + 完整 response	prompt + 已生成的前缀
输出含义	实际多好	预计能拿多少
随 policy 变吗	不变	**变，所以要一直训**

根本差别在最后一行：RM 评的是摆在眼前的完整 response，跟谁写的无关；value function 的预测绑定当前 policy。

Q：为什么要 advantage?

A：因为直接用 reward 当权重**方向是错的**。RM 的分基本都是正数，拿它当权重会把所有 response 的概率都推高，只是好的推得多、差的推得少——差的那条本该被压低。减掉一个「预期」之后，比预期好的变正、差的变负，才有了推和压两个方向。这也和 RM 的性质对上：Bradley–Terry 只约束差值，绝对分本来就没意义。

顺带降方差：权重从 4.5~~5.5 变成 −0.5~~+0.5，缩小一个量级。

Q：为什么要 ratio?

A：因为要**重复用同一批数据**。采样很贵，这批 response 要用 8 次，但 policy gradient 要求数据由当前 policy 生成。ratio 是 importance sampling 的权重，衡量这条数据过期了多少。第 1 个 minibatch 时 ratio 恒为 1，第 8 个时偏离最大。

如果每批数据只用一次，ratio 恒等于 1，也就不需要 clip。ratio 是复用数据的代价，clip 是复用数据的安全绳。

Q：clip 是什么?

A：ratio 乘在梯度上会正反馈爆炸（ratio 大 → 推力大 → ratio 更大）。clip 把 ratio 夹在 [0.8, 1.2]（ε=0.2），超出即截断成边界值，变成常数，**梯度归零**。不是惩罚，是掐断。

配 `min` 之后是**单向**的：往想去的方向走过头就断，往回走随时放行。clip 只拦冲刺，不拦刹车，保证 policy 永远有退路。

Q：完整的损失是什么？是 advantage 吗?

A：不是。advantage 是**权重**（常数，不含 ϕ），损失是权重和 ratio 乘起来再在所有 token 上平均：

```
L_t  = min(ratio_t · A_t, clip(ratio_t, 0.8, 1.2) · A_t)
loss = −(1/N) Σ L_t
```

梯度**全部从 ratio 的分子流出**——A 是常数，π_old 是缓存的常数。

Q：各步的粒度?

A：中间所有量都是 per token，**梯度更新是 per minibatch，一个 batch 更新 8 次**。

操作	粒度
RM 打分	per response（一条一个标量）
KL 惩罚	per token
value 预测	per token
advantage / ratio / clip	per token
梯度更新	**per minibatch**

RM 那一笔**全部落在最后一个 token**，前面的 token 只有接近 0 的 KL 小额扣分。

3.5 PPO-ptx

Q：ptx 是什么?

A：pretraining mix（预训练混合）。PPO 是 γ=0，PPO-ptx 是 γ=27.8。论文里说的 InstructGPT 默认指 PPO-ptx。

Q：它解决什么问题?

A：PPO 训完之后，模型在一批**和训练目标完全无关**的公开数据集上成绩掉了（SQuADv2、DROP、HellaSwag、WMT15 法译英）。论文叫它 **alignment tax（对齐税）**。原因是 PPO 的信号只有 RM 在 API 分布上给的分，而那个分布里分类和 QA 只占约 18%，其余能力没有任何东西在维护，被挤掉了。

在乎它的理由很实际：**有税就会激励人们去用未对齐但更强的模型**。

Q：它加在哪个式子里？是加在打分上吗?

A：**不进打分。** 这是式 (2) 最容易误读的地方——它写成加法，但第三项和前两项不在同一条通路上。第三项里**没有 response**，x 是从预训练语料随机抽的网页文本，和这一轮的 512 条 response 毫无关系，没法加到打分上。

```
通路一：512 条 response → RM + KL → A → ratio → clip → 梯度 ┐
                                                            ├→ 更新 φ
通路二：预训练网页文本 → 语言建模 → ×27.8 → 梯度            ┘
```

唯一的交汇点是梯度缓冲区。 一句话：KL 惩罚**改奖励**，ptx **改梯度**。

Q：实现细节?

A：预训练样本数 = RL episode 数的 **8 倍**；对每个 minibatch **连续两步**分别算 PPO 梯度和预训练梯度，都累加进梯度缓冲区（不是把损失加起来算一次）。位置在第 3 步，第 4 步一起更新。

Q：效果?

A：所有数据集上的退化都被缓解，HellaSwag 甚至超过 GPT-3；DROP、SQuADv2、翻译仍落后。代价是大规模上偏好分略低于纯 PPO。

关键对照：调 γ 能修好退化且几乎不损失验证奖励；**调大 β 修不好**，只会让验证奖励大跌且在 DROP/SQuAD 上从未恢复。原因是 KL 的参照物是 SFT 模型，而 SFT 自己也已经不是预训练分布了。

3.6 评测方法

Q：胜率怎么算?

A：以 **175B SFT** 为共同基线，数某模型的 response 在多少比例的 prompt 上排在它前面。选它是因为**性能处在中游**，50% 即打平。

Q：怎么保证公平?

A：留出 prompt 只选训练时没用过的**客户**。另外论文自己承认训练 prompt 是为 InstructGPT 设计的、对 GPT-3 不公平，所以**另外**在提交给 GPT-3 的 prompt 上重测一遍——这就是 Figure 3 左右两栏。纵向是训练标注员 vs 留出标注员，四个格子结论一致。

Q：GPT (prompted) 是什么模型?

A：**就是 GPT-3 本身**，权重没动，只在前面拼了一段精心设计的 few-shot 提示把它引导进指令跟随模式。它的存在是为了扣掉「对方根本没在答题」那部分差距。加了它之后 InstructGPT 的胜率从 85% 降到 71%。

注意 Figure 9 的失败案例里 GPT-3 **没有加任何提示**，论文明说那不能代表它真实的问答能力——定量比较和定性举例用的基线不一样。

---

4 结果

实验分三节，但其实只有两个数据来源：真实客户 prompt（§4.1，人评）和公开 NLP 数据集（§4.2，自动指标）。§4.3 不出数字，直接展示模型输出。

4.1 API 分布

Q：结果怎么样?

A：五块。

结论	数字
175B InstructGPT vs 175B GPT-3	胜率 **85 ± 3%**
175B InstructGPT vs few-shot GPT-3	胜率 **71 ± 4%**
1.3B PPO-ptx vs 175B GPT-3	**前者胜**（参数少 100 倍以上）
RM 预测留出组标注员偏好	69.6 ± 0.9%（组内 72.4 ± 0.4%）
175B InstructGPT vs FLAN / T0	78 ± 4% / 79 ± 4%

阶梯形状：GPT-3 < GPT-3(prompted) < SFT < PPO ≈ PPO-ptx。

细粒度指标（Figure 4）四个维度全面占优：更常尝试执行正确指令、更遵守显式约束、闭域幻觉更少、用词更适合客服。论文有一句实话：**其他 metadata 类别在 API 上出现太少，得不到统计显著差异**。

Q：FLAN / T0 那个对照说明了什么?

A：同一个 175B GPT-3，在公开指令数据集上各微调约 100 万样本，结果**连 SFT 基线都不如**。两个原因：公开数据集为便于自动评测而设计，偏向分类和 QA（真实用量里只占约 18%，而开放式生成+头脑风暴占约 57%）；以及公开数据集很难覆盖真实输入的多样性。

这条说明**数据来源比方法更关键**，直接塑造了后续所有对话模型的数据策略。

4.2 公开 NLP 数据集

Q：用了哪些数据集?

正文只点名了前八个，完整清单在附录 D：

数据集	测什么	正文提到
TruthfulQA	真实性	是
RealToxicityPrompts	毒性	是
Winogender / CrowS-Pairs	偏见	是
SQuADv2 / DROP / HellaSwag / WMT15 Fr→En	能力退化	是

数据集	测什么	正文提到
QuAC / SST / RTE / WSC	问答、分类、蕴含、指代	否
CNN-DailyMail / Reddit TLDR	摘要	否

全部结果在 Table 14，零样本画在 Figure 28、少样本画在 Figure 29。采样统一在温度 T=0 下做并在第一个换行处截断；多选题按平均每 token 对数概率最低的选项判定。

注意 Table 14 里 CNN-DM 和 TLDR 两行的 ROUGE-L 数值**完全相同**，几乎肯定是排版错误，这两行不能引用。

Q：真实性结果?

A：PPO 模型小幅但显著优于 GPT-3，而且是**默认行为**，不需要专门提示。例外是 1.3B PPO-ptx 略差于同规模 GPT-3，论文没解释。只在非对抗性子集上测，结论仍成立，绝对提升掉几个百分点。给「不确定就说 I have no comment」的提示时，PPO 模型倾向**真实但无信息**而非自信编造。闭域幻觉率 21% vs GPT-3 的 41%。

Q：毒性结果?

A：分三种提示条件看，这是关键结构。

- **respectful prompt**：InstructGPT 毒性更低（自动与人工评测一致）
- **no prompt**：优势消失，两者相当
- **明确要求输出有毒内容**：InstructGPT **比 GPT-3 毒得多**

合起来说明的不是「更无害」，而是「**更听话**」。这一点在 §5.3 被点名为全文最大局限。

采样上有个调整：按 prompt 毒性**均匀采样**（非该数据集标准做法），所以绝对毒性数字被抬高了，不能横向比。

附带发现：SFT 基线毒性最低，但连贯性最低、排序里最不受偏好——可能只是因为它生成得很短或已退化。单看毒性最好的模型，可能只是什么都没好好说。

Q：偏见结果?

A：**没有改善**。用 Winogender 和 CrowS-Pairs 的成对句子算相对概率与熵，无偏模型应当熵最大。PPO-ptx 偏见与 GPT-3 相当；被要求 act respectfully 时熵反而**下降**（偏见更强）。论文说模式不清晰，解读是：指令化的模型对自己的输出更确定，不管输出是否体现刻板印象。

4.3 定性结果

Q：这一节展示了多少例子？

A：正文只有 Figure 8 和 Figure 9 共四个（法语写青蛙故事、解释二项式系数代码、吃袜子的假前提、炮弹打南瓜的过度对冲），附录 Figure 42–49 还有十来个。

Q：泛化的部分说明了什么？

A：非英语和代码在微调数据里占比极小（数据 96% 以上是英文），但模型仍能跟随非英语指令、对代码做摘要问答。这表明对齐方法**可以泛化到人类没有直接监督的输入上**。短板是常常仍用英语作答，而且论文明说**没有做定量跟踪**——这一节是观察，不是测量。

Q：三类简单错误？

A：

1. **假前提**——顺着错误假设往下说
2. **过度对冲**——简单问题也说没有唯一答案
3. **多重约束下性能下降**

第 2 类的成因论文追到了标注指南：**指示标注员奖励认知谦逊（epistemic humility），标注员倾向给对冲的输出更高排名，这被 RM 学了过去**。这是全文唯一一条把具体缺陷一路追溯到标注规则的因果链——不是模型学坏了，是一条合理的规则被 RM 拟合成了「含对冲措辞就加分」。

论文认为前两类都能通过**对抗式数据收集**大幅减少。

---

5 讨论

这一节**没有实验**。消融全部在附录 E，正文只引用结论。章节编号里 5.3（局限）因标题格式不同没进目录，内容还在。全文**没有结论章节**，最后一句落在 §5.5。

Q：对齐研究的四条启示？

A：最有分量的是第一条——**对齐成本相对预训练很低**。175B SFT 约 4.9 petaflops/s-days，175B PPO-ptx 约 60，而 GPT-3 预训练是 3,640，即 PPO-ptx 约为预训练的 **1.6%**，效果却超过把模型放大 100 倍。其余三条：对齐在无监督场景（代码、非英语）有泛化迹象，这很重要因为让人类监督每个任务不可行；低对齐税是技术能否被采纳的关键；以及把偏理论的对齐研究落到了真实生产系统上。

Q：「我们在对齐谁」讲了什么？

A：全文最坦诚的一节。实际对齐的是四层：约 40 名标注员的偏好（多为居住在美国或东南亚的英语使用者，彼此约 27% 的时候不一致）；研究者自身的偏好（标注指南和答疑都是作者写的）；OpenAI 客户认为有价值的用途（客户目标未必与其终端用户利益一致）；而这批客户还来自以 OpenAI 员工为种子的候补名单。

作者明确表示不主张这三方是偏好的「正确来源」，并指出**不可能训练出同时对齐所有人偏好的系统**，一条可能路径是训练可按群体偏好条件化的模型。

Q：局限？

A：方法上，标注团队小且不具代表性；**多数比较只由 1 人标注**（成本原因），无法识别分歧区域，而在分歧场景下「对齐到平均偏好」未必可取。模型上，既不完全对齐也不安全。论文自己点名的**最大局限**是：模型基本会照做用户指令，**即使该指令会导致现实危害**——这正好和 §4.2 毒性那三种提示条件对上。

Q：开放问题里哪些后来成了主线？

A：**训练模型拒绝有害指令**（→ RLHF safety、Constitutional AI）；**比较排序未必是最高效的反馈形式**，可让标注员直接编辑回复或写自然语言批评（→ critique、RLAIF）；PPO 之外的算法，如 expert iteration、在比较数据子集上做 behavior cloning、约束优化（→ DPO）；以及对抗式数据收集、过滤预训练数据里的有毒内容。

附：复现可行性与最小实验方案

Q：这篇论文能复现吗？

A：分三层。

严格复现不可能，四个卡点：训练数据是真实客户 prompt，涉及隐私，从未公开；标注员写的 demonstration 和几十万个比较对也没公开；底座 GPT-3 权重没开放；评测要约 40 名标注员对 3,196 条 prompt 上十几个模型的输出排序。

方法复现完全可行。附录 C 给了几乎全部超参，附录 B 给了标注指南节选和标注员筛选流程，§3.5 两个式子完整。论文没有代码仓库链接，但实现上没有信息缺口。

换组件跑通这条路是通的：底座换开源模型，数据换公开偏好数据集，代码用现成框架。注意超参不能照搬——β 尤其不能，RM 输出尺度一变它就不对了。

Q：3090 上的最小实验方案？

A：目的是**看清现象，不是做出好模型**。

显存估算（bf16）：训练中的模型约 16P 字节（参数 2P + 梯度 2P + Adam 约 12P），冻结的约 2P。**0.5B 底座**：policy 全量微调约 8GB，reference 1GB，RM 1GB，value function 挂在 policy 骨干上共享参数，静态开销约 10GB 出头，剩余留给 KV cache 和激活值。所以 0.5B **可以全量微调，完整复刻四模型结构**，比上 LoRA 更干净。

任务推荐 **TL;DR 摘要**（Stiennon 2020 的原任务）：SFT 数据和比较数据都公开且同分布；输出短，生成快；而且**长度偏置极显眼**，是验证 over-optimization 最省事的途径。对话类数据集不建议作第一个任务。

四个阶段：

1. **SFT** — 只在 response 部分算 loss。跑完**存两份权重**：一份当 policy 起点，一份冻结当 reference。验收：能生成通顺的摘要即可。
2. **RM** — 改标量头，Bradley-Terry，1 epoch 早停。验收：留出集成对准确率 65–70%。训完照论文做归一化，让 SFT 生成的摘要平均得分为 0。
3. **PPO** — 工作量大头，难点全在论文没写的地方：四模型显存编排、生成与训练阶段切换、采样时缓存旧 policy 的 per-token 概率、per-token 的 KL 与 advantage 对齐、minibatch 切分不要切断 response。
4. **消融** — 整件事的价值所在。

起始配置抄论文，只缩小 batch：

项	论文	3090
batch（prompt 数）	512	32 或 64
minibatch	64（8 个）	8（4 个）
inner epoch	1	1
clip ε	0.2	0.2
KL 系数 β	0.02	先试 0.02，**很可能要调**
采样温度	1	1
GAE	不折扣	不折扣，λ 先取 0.95

β 的判断方法：KL 曲线涨到几十说明太小；KL 贴着 0 不动而 reward 也不涨说明太大。

三个必做消融：把 **β 设成 0** 重跑，看 reward 一路上涨而输出崩坏；**长度分析**，把平均 response 长度和 reward 曲线叠在一起看；**ptx**，在留出预训练语料上测困惑度，纯 PPO 跑久了会升，加 ptx 后应稳住。

必须记录的曲线（论文一个字没提，但没有这些什么都看不出来）：平均 reward（扣 KL 之前）、平均 KL、平均 response 长度、**clip 比例**（健康值百分之几到十几，超三成说明每步走太猛）、value loss 与 **explained variance**（长期接近 0 说明 advantage 基本是噪声）、token 熵（塌缩前兆是熵急剧下降）。另外每隔若干步**固定几条 prompt 打印生成结果**——曲线会骗人，输出不会。